



# **UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO**

## **FACULTAD DE INGENIERÍA**

**Proyecto: Desarrollo del Videojuego Pac-Man en Lenguaje C**

**Asignatura: Fundamentos de Programación**

**Equipo: Las cabras de progra**

**Profesora: Ariel Adara Mercado Martinez**

**Integrantes:**

**Carbajal Sierra José Alfredo**

**Hernández López Samuel Noam**

**Ordoñez Reveles Héctor Eduardo**

**Villaseñor Juárez Iván**

**Semestre: 2026-2**

**Fecha de Entrega: 12 de junio de 2026**

## **Distribución de Tareas**

Para avanzar de forma organizada y aprovechar al máximo el trabajo en equipo, decidimos dividir el desarrollo del videojuego entre nuestros integrantes de manera equitativa, asignando a cada uno responsabilidades específicas sobre estos cinco grandes componentes lógicos:

- **Módulo de Renderizado y Gráficos (SDL2):** Se encarga de gestionar la ventana principal, cargar las texturas necesarias y realizar el dibujo de cada elemento en pantalla.
- **Módulo de Control y Movimiento del Jugador:** Gestiona la interacción con el teclado, controla la animación del personaje y calcula su posición exacta, además de incluir la lógica de colisiones con los bordes de la pantalla y el efecto de túnel.
- **Módulo del Sistema de Mapas:** Administra la estructura del laberinto mediante matrices y gestiona la lógica de los elementos visuales del escenario.
- **Módulo de Inteligencia de Fantasmas:** Coordina el comportamiento de los enemigos, desde su lógica de salida y movimiento, hasta el modo de persecución cuando detectan al jugador y la detección de colisiones que restan vidas.
- **Módulo de Menús, Archivos y Puntuaciones:** Supervisa el flujo entre las distintas pantallas del juego, controla la persistencia de datos mediante la lectura y escritura de archivos de texto y mantiene actualizados los récords de los jugadores.

## **Avances del Proyecto**

Para avanzar de forma organizada y aprovechar al máximo el trabajo en equipo, decidimos dividir el desarrollo del videojuego entre nuestros cinco integrantes de manera equitativa. El proceso se estructuró en etapas clave, comenzando por una fase de análisis de problemas donde desglosamos cada componente tanto a nivel general como en sus funciones específicas, lo que nos permitió entender las necesidades técnicas de cada área desde el principio.

Una vez comprendido el alcance, pasamos a la etapa de diseño, donde dedicamos tiempo a elaborar pseudocódigos y diagramas de flujo. Esto fue fundamental para planificar la arquitectura lógica del sistema y los flujos de control, evitando

contradicciones antes de escribir la primera línea de código en C.

Posteriormente, entramos en la fase de codificación, donde cada integrante desarrolló una propuesta funcional para su módulo asignado; este enfoque modular permitió que cada uno se especializara en su área, ya sea en la lógica de renderizado, el movimiento del jugador, la estructura del mapa, la inteligencia de los fantasmas o la gestión de archivos. Finalmente, realizamos el proceso de integración, un paso crítico donde conectamos cada propuesta en el archivo principal (main). Durante esta fase, trabajamos de manera colaborativa realizando ajustes constantes, depurando errores de sintaxis y lógica, y asegurando que las funciones interactuaran de forma coherente hasta lograr que el proyecto compilara y ejecutara sin conflictos.

## **Problemas Detectados y Soluciones**

A lo largo del proyecto, nos enfrentamos a desafíos técnicos que fueron escalando en complejidad conforme avanzábamos. Las etapas iniciales fueron de un rigor intelectual alto: pensar en las mejores soluciones algorítmicas, plasmarlas en pseudocódigos estructurados y, posteriormente, llevar toda esa lógica al lenguaje C fue un proceso que puso a prueba nuestra capacidad de análisis y síntesis. Sin embargo, aunque estas fases fueron demandantes, resultaron ser la parte sencilla al compararlas con la pesadilla logística que significó la integración del proyecto.

El proceso de unir nuestros módulos fue, sin lugar a dudas, la etapa más crítica y desgastante. Al intentar ensamblar las piezas, nos encontramos con un caos inesperado: las funciones no encajaban entre sí porque los criterios de diseño variaban entre integrantes, y existía una falta de claridad sobre qué variables y punteros debían compartirse entre los distintos bloques de código. Esto provocaba constantes errores de desfase, comportamientos erráticos o, directamente, el fallo total del programa al intentar compilar.

Para salir de este estancamiento, adoptamos una estrategia mucho más metódica. Decidimos dejar de intentar integrar todo de golpe y empezamos a "apilar" las funciones una por una, probando rigurosamente qué piezas encajaban con qué

estructuras. Fue un ejercicio de paciencia extrema donde, línea a línea, íbamos depurando dependencias y ajustando los parámetros para que el sistema finalmente tuviera sentido. Aprendimos que, en la programación colaborativa, el código no solo debe ser funcional de manera aislada, sino que debe tener una coherencia global; aunque el resultado final no fuera perfecto ni impecable, este proceso de integración manual y constante nos permitió entender la arquitectura completa de nuestro videojuego y lograr, tras mucho esfuerzo, que todas las piezas trabajaran de forma conjunta.

## **Uso de Inteligencia Artificial (IA)**

Durante el proyecto, utilizamos la inteligencia artificial como una herramienta de apoyo para mejorar la calidad y la organización de nuestro trabajo, pero siempre bajo nuestro control. La IA fue fundamental principalmente para revisar la sintaxis de nuestros programas en C y ayudarnos a unificar el uso de variables y nombres entre los diferentes módulos, asegurando que el código fuera consistente.

Además, nos sirvió para organizar mejor nuestras ideas: usamos la IA para estructurar toda la documentación bajo el formato de Entrada, Proceso y Salida, lo que nos permitió que el documento final fuera coherente, profesional y fácil de leer para todos. También la empleamos como un apoyo para detectar errores específicos en nuestros códigos durante la fase de integración. Es muy importante aclarar que la inteligencia artificial nunca fue utilizada para crear el diseño lógico del juego ni para inventar sus reglas; todas las ideas, mecánicas y estrategias del videojuego fueron propuestas totalmente por nosotros, los integrantes de la brigada.

## **Análisis de problemas**

### **Renderizado y Gráficos**

#### **Análisis del problema general**

Entrada: los datos de los objetos en el juego, la hoja de texturas y el motor de renderizado (SDL2).

Salida: una representación visual fluida y coherente de todos los elementos del juego en la pantalla.

Proceso General:

1. Crear y configurar la ventana y el contexto de renderizado del motor.
2. Cargar las texturas necesarias desde los archivos de imagen a la memoria gráfica.
3. Limpiar el fotograma anterior para preparar el lienzo de dibujo.
4. Dibujar los elementos del juego (mapa, jugador y enemigos) en su posición y orden correcto.
5. Actualizar la pantalla para mostrar el fotograma recién generado al usuario.
6. Inicialización del motor Prepara el entorno gráfico para que el juego pueda mostrarse en pantalla. Entrada: parámetros de configuración (título de la ventana, dimensiones). Salida: una ventana abierta y un motor de renderizado listo para dibujar. Proceso:
7. Inicializar la biblioteca gráfica SDL2.
8. Crear la ventana principal con las dimensiones definidas.
9. Crear el renderizador (renderer) vinculado a la ventana.
10. Carga de texturas Convierte archivos de imagen en recursos que la tarjeta gráfica puede procesar rápidamente. Entrada: la ruta del archivo de imagen y el motor de renderizado. Salida: un puntero a una textura de SDL lista para ser dibujada. Proceso:
11. Cargar la imagen desde el archivo a una superficie temporal.
12. Convertir esa superficie en una textura optimizada para el renderizador.
13. Liberar la superficie temporal para optimizar el uso de memoria.
14. Ciclo de renderizado (Render Loop) Gestiona la secuencia de dibujo para asegurar que los elementos se vean correctamente. Entrada: los objetos activos del juego (mapa, jugador, fantasmas). Salida: el fotograma actualizado en el monitor. Proceso:
15. Limpiar el renderizador con un color de fondo (usualmente negro).
16. Llamar a las funciones de renderizado de cada módulo (mapa, jugador, fantasmas) en el orden correcto para establecer las capas de profundidad.
17. Presentar el fotograma terminado en la pantalla para finalizar la actualización visual.
18. Limpieza y cierre Libera los recursos gráficos al finalizar el juego. Entrada: el puntero a las texturas y al renderizador. Salida: memoria liberada correctamente. Proceso:
19. Destruir las texturas cargadas.

20. Destruir el renderizador y cerrar la ventana.

21. Salir de la biblioteca SDL2 para liberar la memoria del sistema.

## **Player**

### **Análisis del problema general**

Entrada: las pulsaciones del teclado proporcionadas por el usuario y el tiempo de ejecución del sistema.

Salida: el personaje dibujado en la pantalla, desplazándose según la dirección deseada y con la animación correcta.

Proceso General:

1. Asignar la posición y el estado inicial del jugador la primera vez que inicia la partida.
2. Leer los botones de las flechas para definir hacia dónde debe mirar y a qué velocidad debe moverse.
3. Calcular la nueva posición espacial del personaje basándose en su velocidad.
4. Controlar los límites de la ventana para que el jugador reaparezca del lado opuesto si sale por los extremos laterales, o impedir que salga por arriba y abajo.
5. Cambiar los cuadros de la animación dependiendo de si está quieto o moviéndose.
6. Dibujar el personaje actualizado en pantalla.

### **1. Control, movimiento y renderizado**

Maneja toda la lógica del protagonista, desde presionar botones hasta su despliegue visual en pantalla.

Entrada: un apuntador con las propiedades actuales del jugador, el motor de renderizado, la hoja de texturas y el estado de las teclas.

Salida: coordenadas matemáticas alteradas, cambios de cuadro en su animación y la impresión de su gráfico actualizado en la pantalla.

Proceso:

1. Colocar al jugador en el centro matemático de la pantalla, fijar su velocidad en cero, mirar hacia abajo y etiquetar su estado como quieto si es la primera vez que se ejecuta.
2. Reiniciar la velocidad a cero para evitar deslizamientos infinitos y comprobar

- qué tecla direccional se pulsa para inyectarle velocidad positiva o negativa.
3. Sumar las velocidades a su ubicación actual para moverlo.
  4. Evaluar si su coordenada cruza el margen izquierdo o derecho para enviarlo de golpe al lado opuesto del plano, o frenarlo si choca con el techo o el suelo.
  5. Identificar si debe reproducir los fotogramas de caminar o de estar quieto y calcular la rapidez en la que cambian las imágenes según el reloj del juego.
  6. Seleccionar el cuadro de imagen correcto según su dirección y plasmarlo en la posición exacta recién generada en la ventana.

## **Mapas**

### **Análisis del problema general**

Entrada: un archivo de texto con el diseño estructurado del nivel y una imagen de disco para la textura del laberinto.

Salida: el laberinto completamente dibujado en la ventana gráfica con sus muros, pasillos (espacios vacíos) y un botón de cierre interactivo.

Proceso General:

1. Leer el archivo de texto y traducir sus caracteres a una matriz bidimensional.
2. Cargar la imagen del laberinto en la memoria gráfica del motor.
3. Renderizar la textura base en la pantalla.
4. Dibujar cuadros negros en las áreas que la matriz indica como espacios vacíos para simular los pasillos.
5. Pintar un botón rojo con una cruz para cerrar el juego.

### **1. Cargar el mapa de texto**

Lee el archivo de texto para generar el molde geométrico del nivel.

Entrada: el nombre o la ruta del archivo de texto que contiene el mapa.

Salida: una matriz global bidimensional llena con los caracteres correspondientes al diseño del nivel.

Proceso:

1. Abrir el archivo especificado en modo lectura.
2. Recorrer cíclicamente las filas y columnas del tamaño predefinido del mapa.
3. Extraer cada carácter ignorando los saltos de línea (como '\n' o '\r').
4. Guardar los caracteres válidos detectados directamente en la matriz global y cerrar el documento.

## **2. Cargar las imágenes**

Convierte una imagen guardada en la computadora en un formato de textura utilizable por el juego.

Entrada: el motor de renderizado y la ruta del archivo de imagen.

Salida: un valor de verdadero si la textura cargó correctamente, o falso si ocurrió un fallo, además de guardar la textura en la memoria.

Proceso:

1. Cargar la imagen cruda en una superficie temporal.
2. Crear la textura gráfica definitiva utilizando la superficie temporal.
3. Liberar la memoria de la superficie temporal para no desperdiciar recursos.
4. Verificar que la textura no esté vacía y devolver confirmación de éxito.

## **3. Renderizar el nivel**

Dibuja los elementos estáticos del nivel sobre la ventana de juego.

Entrada: el motor de renderizado.

Salida: gráficos plasmados y visibles en la pantalla.

Proceso:

1. Dibujar la textura completa del laberinto como fondo base.
2. Recorrer la matriz de coordenadas del mapa celda por celda.
3. Calcular las medidas si se detecta un espacio vacío y dibujar un cuadrado completamente negro para tapar el fondo y crear el pasillo.
4. Dibujar un rectángulo rojo con un par de líneas cruzadas en la esquina superior para simular un botón de cierre.

## **Fantasmas**

### **Análisis del problema general**

**Entrada:** el tiempo que lleva corriendo la partida, la posición exacta del jugador y el mapa.

**Salida:** movimiento automatizado de hasta 4 fantasmas, si tocan al jugador, que este pierda una vida.

**Proceso General:**

1. Inicializar a los 4 enemigos manteniéndolos encerrados en su posición inicial.
2. Evaluar el tiempo transcurrido para liberar a los fantasmas uno por uno de



forma automatizada.

3. Calcular el movimiento continuo de los fantasmas que ya están afuera.
4. Detectar colisiones contra las paredes del mapa para hacer que cambien de dirección al azar.
5. Medir la distancia entre el jugador y los enemigos para activar el modo de persecución si pasan muy cerca.
6. Verificar si las coordenadas de un fantasma y el jugador se cruzan para detener el juego, registrar la captura y restar una vida.

## **Análisis de las funciones**

### **1.Posición inicial**

Asegurarse de que un fantasma tenga una posición inicial.

Entrada: nada o 0.

Salida: coordenadas de los fantasmas (la caja).

Proceso:

1. Asignar la coordenada horizontal (X) correspondiente a la caja central del laberinto.
2. Asignar la coordenada vertical (Y) correspondiente a la caja central del laberinto.

### **2. Activación de los fantasmas**

Activa a un fantasma para que empiece a jugar.

Entrada: el número de fantasma que toca (dependiendo de cuántos hay activos en el juego).

Salida: propiedades de movimiento asignadas (velocidad, dirección y propiedad de que está activo en el juego).

Proceso:

1. Recibir el número de identificación del fantasma a activar.
2. Asignar una velocidad inicial de salida.
3. Establecer la dirección inicial hacia arriba para que salga de la caja.
4. Cambiar la etiqueta del fantasma a "activo" para que el juego sepa que debe empezar a dibujarlo y moverlo.

### **3. Control de salida**

Deja salir a los fantasmas poco a poco.

Entrada: un reloj, el conteo de cuántos han salido y el tiempo que deben esperar.

Salida: un fantasma nuevo suelto en el mapa.

Proceso:

1. Revisar el tiempo actual en el reloj del juego.
2. Comparar el tiempo actual con el tiempo registrado en el que salió el último fantasma.
3. Si la diferencia es mayor o igual al tiempo de espera programado, llamar al siguiente fantasma.
4. Sumar uno al contador total de fantasmas sueltos.
5. Actualizar el registro del cronómetro con la hora de salida de este nuevo fantasma.

#### **4. Movimiento y colisión**

Hace caminar a los fantasmas y evita que atraviesen los muros.

Entrada: dónde está el fantasma, hacia dónde mira y el plano del laberinto.

Salida: movimiento o un cambio de dirección.

Proceso:

1. Calcular mentalmente en qué cuadro caerá el fantasma sumando un paso en su dirección actual.
2. Evaluar si en esa posición futura existe un muro dentro de la matriz del mapa.
3. Si se detecta un muro, elegir aleatoriamente una nueva dirección (arriba, abajo, izquierda o derecha).
4. Si el camino está despejado, actualizar las coordenadas del fantasma para que avance a esa posición futura.

#### **5. Persecución**

Le da a los fantasmas la capacidad de detectar al jugador y perseguirlo si está cerca.

Entrada: la posición del jugador y la ubicación de todos los fantasmas sueltos.

Salida: cambio de dirección hacia el objetivo.

Proceso:

1. Medir la distancia que existe entre cada fantasma libre y el jugador.
2. Evaluar si esta distancia es menor a 5 cuadros (zona de visión).

3. Si el jugador está dentro de la zona, comparar en qué coordenadas se encuentra respecto al enemigo.
4. Cambiar la dirección del fantasma para que coincida con la del jugador (ej. si el jugador está a la derecha, girar a la derecha para intentar interceptarlo).

## **6. Captura**

Avisa si el jugador perdió.

Entrada: la posición del jugador y la posición de los fantasmas.

Salida: una respuesta de verdadero (atrapado) o falso (a salvo).

Proceso:

1. Medir la distancia exacta entre el del jugador y el de cada fantasma que esté suelto en el mapa.
2. Evaluar si esa distancia llega a ser cero (se enciman).
3. Si se encima, interrumpir las verificaciones y enviar una respuesta de "verdadero" para que el jugador pierda una vida.
4. Si se revisa a todos los fantasmas y nadie toca al jugador, enviar una respuesta de "falso" para que el juego continúe.

## **7. Reinicio**

Limpiar el mapa cuando el jugador pierde una vida para volver a empezar.

Entrada: ninguna.

Salida: contadores de tiempo y enemigos en cero.

Proceso:

1. Devolver el contador de fantasmas activos a cero.
2. Reiniciar el cronómetro que controla el tiempo de las salidas.
3. Cambiar la etiqueta de todos los fantasmas a "inactivos" para esconderlos de la pantalla.
4. Regresar a todos los fantasmas a su punto de origen en la caja.

## **Menús, Archivos y Puntuaciones**

### **Análisis del problema general**

Entrada: archivos de texto con configuraciones o récords y las acciones del usuario en pantalla.

Salida: un flujo de navegación entre menús, actualización de archivos persistentes y despliegue de información de puntaje.

Proceso General:

1. Gestionar las transiciones entre las distintas pantallas (menú principal, juego, game over).
2. Leer información desde archivos externos para cargar configuraciones o puntajes previos.
3. Escribir nuevos datos en archivos para guardar los récords alcanzados.
4. Procesar la lógica de interacción del usuario para navegar en los menús.
5. Gestión de estados de pantalla Controla qué interfaz se muestra al usuario en cada momento del juego. Entrada: el estado actual del juego. Salida: la interfaz correspondiente en pantalla. Proceso:
6. Verificar si el estado requiere el menú principal, la partida en curso o la pantalla de resultados.
7. Renderizar únicamente los elementos visuales y detectar las entradas de teclado válidas para ese estado específico.
8. Carga y guardado de archivos Garantiza que la información del juego persista aunque se cierre la aplicación. Entrada: la ruta del archivo de texto. Salida: lectura de datos hacia variables del juego o escritura de datos desde variables al archivo. Proceso:
9. Abrir el archivo en modo lectura para obtener los datos necesarios al iniciar.
10. Procesar el formato del archivo para asignar los valores a las estructuras de datos correspondientes.
11. Abrir el archivo en modo escritura para actualizar los récords al finalizar la partida o cuando ocurra un evento relevante.
12. Procesamiento de puntuaciones Calcula y mantiene el seguimiento del desempeño del jugador. Entrada: eventos del juego (consumo de objetos, tiempo). Salida: valor numérico de puntuación actualizado. Proceso:
13. Sumar puntos según las reglas definidas al interactuar con elementos del mapa.
14. Comparar el puntaje actual con el récord almacenado en el archivo.
15. Actualizar el récord si el puntaje actual es superior.

Pruebas

